

计算几何专题训练

原作者：memset0，魔改完善：EH

2025-09-20



01

引言



1.1 精度建模

IEEE-754 提供的精度保障:

能精确表示的整数:

- 对于 double, 约为 9×10^{15} .
- 对于 long double, 约为 1.8×10^{19} .
- 特别地, long double 类型可以精确执行所有 long long 类型可以执行的计算.

浮点计算误差:

- 每次 $+$, $-$, $\times$, $/$, $\sqrt{\cdot}$ 等操作都可能引入 **相对误差**, 记为 ε (machine epsilon) .
- 对于 double, $\varepsilon < 1.2 \times 10^{-16}$.
- 对于 long double, $\varepsilon < 5.5 \times 10^{-20}$.

提醒: 这是相对误差.

1.1 精度建模

一些常见的爆炸情况：

- 一堆数加加减减，先加一个大数最后再减掉，则相对于“最后结果”的相对误差会特别大.
- 加减法累积出的误差，被乘除法加倍放大 (e.g. 小心 $1/x$) .
- 一些小误差导致 $\sqrt{\cdot}$ 、 $\text{acos}(\cdot)$ 等函数越过定义域而返回 nan.

Keeping the Dogs Apart (CF101550K)

有两只狗 A 和 B ，分别沿着折线 $A_0, A_1, \dots, A_n$ 和 $B_0, B_1, \dots, B_m$ 以相同的速度移动。求两只狗之间的最近距离。

$1 \leq n, m \leq 10^5$ ，精度误差 10^{-4} 。

还是很好想到需要用双指针扫一下的。

问题简化为做 $n + m$ 次：一只狗在线段 $|PQ|$ 上移动，另一只狗在线段 $|RS|$ 上移动，求最短距离。记为 $\text{minDist}(p, q, r, s)$ 函数（这个函数需要求导去做）

时间复杂度 $O(n + m)$ 或如果使用三分 $O((n + m) \log V)$ 。

1.2 案例研究

1.2.1 看起来很对但过不去的代码

```
int i = 0, j = 0; // current segment of A and B
double ans = absv(a[0]-b[0]), // closest distance so far
        sumA = 0, // total length of segments fully walked by A
        sumB = 0; // total length of segments fully walked by B

// While both dogs are still walking
while (i+1 < n && j+1 < m) {
    double start = max(sumA, sumB), // start of time interval
           dA = absv(a[i+1]-a[i]), // length of current segment of A
           dB = absv(b[j+1]-b[j]), // length of current segment of B
           endA = sumA + dA, // time at which A will end this segment
           endB = sumB + dB, // time at which B will end this segment
           end = min(endA, endB); // end of time interval

    // Compute start and end positions of both dogs
```

1.2 案例研究

```
pt p = moveBy(a[i], a[i+1], start-sumA),
    q = moveBy(a[i], a[i+1], end-sumA),
    r = moveBy(b[j], b[j+1], start-sumB),
    s = moveBy(b[j], b[j+1], end-sumB);

// Compute closest distance for this time interval
ans = min(ans, minDist(p,q,r,s));

// We get to the end of the segment for one dog or the other,
// so move to the next and update the sum of lengths
if (endA < endB) {
    i++, sumA += dA;
} else {
    j++, sumB += dB;
}
}
```

1.2 案例研究

1.2.2 为什么会这样呢？

对于 `sumA` 和 `sumB` 这两个变量：

- 最多进行 $k = 2 \times 10^5$ 次加法运算.
- 值域最大为 $M = \sqrt{2} \times 10^9$.
- 假设使用 `double`, 则 $\varepsilon = 1.2 \times 10^{-16}$.

故绝对误差最大为 $kM\varepsilon \approx 0.032$, 超过了 10^{-4} 的精度要求, 导致 Wrong Answer.

1.2 案例研究

1.2.3 巧解

此题中，注意到只有 $\text{sumA} - \text{sumB}$ 是有用的。或者说，同时给 sumA 和 sumB 加上一个常数，不影响计算。

因此，可以在 while 循环体之后加上：

```
double minSum = min(sumA, sumB);  
sumA -= minSum;  
sumB -= minSum;
```

通过限制值域来减小绝对误差。

不过这并不是一个通用方法。

1.3 浮点数求和

1.3.1 如何更精确地进行浮点数求和？

刚才的问题中，我们进行了 k 次加法运算，导致相对误差最大为 $k\varepsilon$ 。

这是因为我们的加法实际上是这样做的：

$$((((a_1 + a_2) + a_3) + a_4) + \dots) + a_k$$

能否用类似二叉树的结构，把这些数加起来？以 $k = 8$ 为例：

$$(((a_1 + a_2) + (a_3 + a_4)) + ((a_5 + a_6) + (a_7 + a_8)))$$

这样“求和次数”就降低为了 $\log k$ 次，相对误差就只有 $\log(k) \cdot \varepsilon$ 了。

刚才那题，使用这一 trick 将绝对误差从 $kM\varepsilon$ 优化成 $\log(k)M\varepsilon$ 也能通过。

1.3 浮点数求和

1.3.2 封装好的模板

```
struct stableSum {  
    int cnt = 0;  
    vector<double> v, pref{0};  
    void operator+=(double a) {  
        assert(a >= 0); // make sure a is non-negative, otherwise...  
        int s = ++cnt;  
        while (s % 2 == 0) {  
            a += v.back();  
            v.pop_back(), pref.pop_back();  
            s /= 2;  
        }  
        v.push_back(a), pref.push_back(pref.back() + a);  
    }  
    double val() { return pref.back(); }  
};
```



02

二维计算几何模板

2.1 点

2.1.1 基本定义

```
typedef long double T;
struct pt {
    T x, y;
    pt operator+(pt p) {return {x+p.x, y+p.y};}
    pt operator-(pt p) {return {x-p.x, y-p.y};}
    pt operator*(T d) {return {x*d, y*d};}
    pt operator/(T d) {return {x/d, y/d};} // only for floating-point
    bool operator==(const pt& b){return x == b.x && y == b.y;}
    bool operator!=(const pt& b){return !(*this == b);}
};
T sq(pt p) {return p.x*p.x + p.y*p.y;}
T absv(pt p) {return sqrt(sq(p));}
```

2.1 点

2.1.2 辅助函数

可以使用 `std::ostream` 来定义点的输出方式，方便配合 `std::cerr` 进行调试.

```
ostream& operator<<(ostream& os, pt p) {  
    return os << "(" << p.x << ", " << p.y << ")";  
}
```

用法:

```
pt a = {3,4}, b = {2,-1};  
cout << a+b << " " << a-b << "\n"; // (5,3) (1,5)  
cout << a*-1 << " " << b/2 << "\n"; // (-3,-4) (1,-0.5)
```

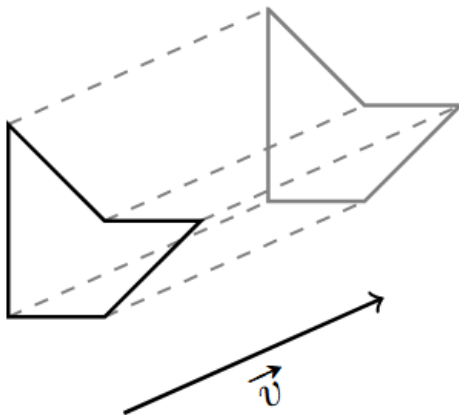
定义 `sgn` 函数将值域中的数映射到 $\{-1, 0, 1\}$ 上.

```
int sgn(T x) { return (T(0) < x) - (x < T(0)); }
```

2.1 点

2.1.3 基本变换：平移

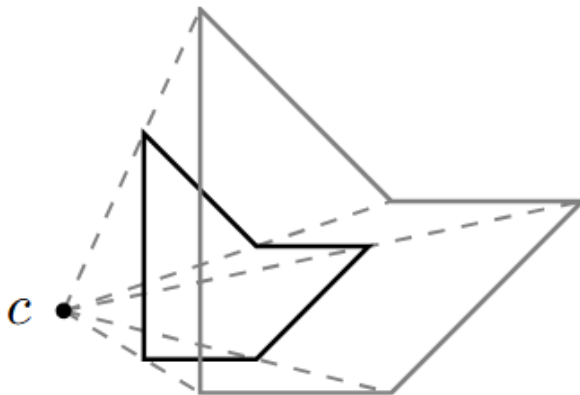
```
pt translate(pt v, pt p) {  
    return p + v;  
}
```



2.1 点

2.1.4 基本变换：缩放

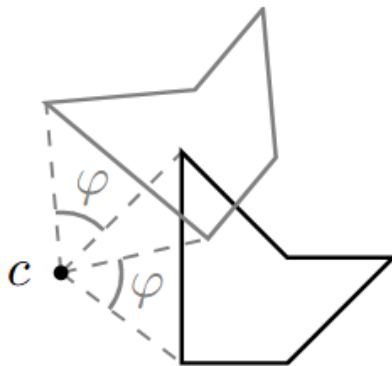
```
pt scale(pt c, T factor, pt p) {  
    return c + (p - c) * factor;  
}
```



2.1 点

2.1.5 基本变换：旋转（逆时针）

```
pt rot(pt p, T a) {  
    return {p.x * cos(a) - p.y * sin(a), p.x * sin(a) + p.y * cos(a)};  
}  
  
pt perp(pt p) {  
    return {-p.y, p.x};  
}
```



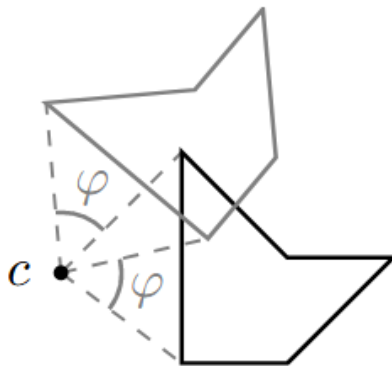
2.1 点

对极坐标熟悉的同学，可以从复数的角度推出这一公式. (试试看?)

复数乘法：模长相乘，辐角相加.

逆时针旋转 φ 相当于乘以 $\text{cis } \varphi = \cos \varphi + i \sin \varphi$.

$$\begin{aligned}(x + yi) * \text{cis } \varphi &= (x + yi) * (\cos \varphi + i \sin \varphi) \\ &= (x \cos \varphi - y \sin \varphi) + (x \sin \varphi + y \cos \varphi)i\end{aligned}$$

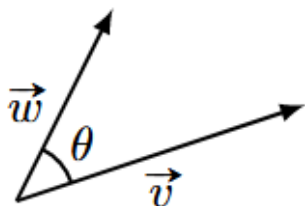


2.1 点

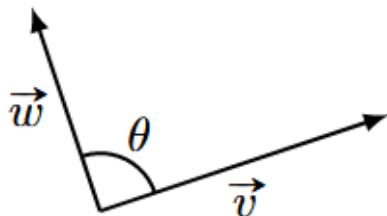
2.1.6 点积

$$\vec{v} \cdot \vec{w} = \|\vec{v}\| \|\vec{w}\| \cos \theta$$

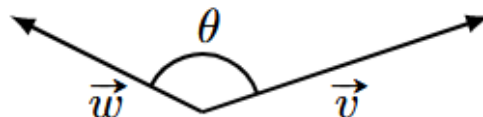
```
T dot(pt v, pt w) { return v.x * w.x + v.y * w.y; }  
bool isPerp(pt v, pt w) { return dot(v, w) == 0; }
```



$$\theta < \pi/2$$
$$\vec{v} \cdot \vec{w} = 5$$



$$\theta = \pi/2$$
$$\vec{v} \cdot \vec{w} = 0$$



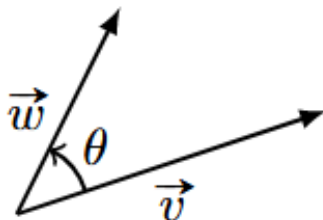
$$\theta > \pi/2$$
$$\vec{v} \cdot \vec{w} = -5$$

2.1 点

2.1.7 叉积

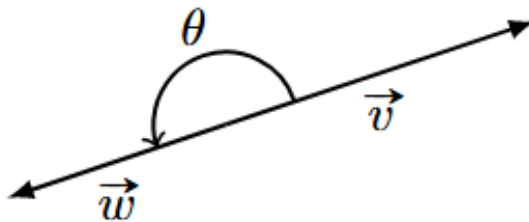
$$\vec{v} \times \vec{w} = \|\vec{v}\| \|\vec{w}\| \sin \theta$$

```
T cross(pt v, pt w) { return v.x * w.y - v.y * w.x; }
```



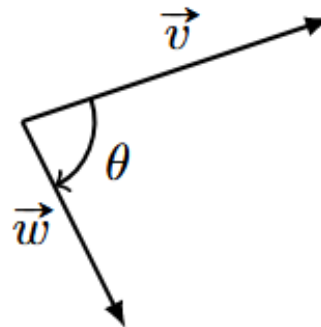
$$0 < \theta < \pi$$

$$\vec{v} \times \vec{w} = 5$$



$$\theta = \pi$$

$$\vec{v} \times \vec{w} = 0$$



$$-\pi < \theta < 0$$

$$\vec{v} \times \vec{w} = -7$$

2.1 点

可以用点积求向量夹角.

```
T angle(pt v, pt w) {  
    T cosTheta = dot(v,w) / absv(v) / absv(w);  
    return acos(max(-1.0, min(1.0, cosTheta)));  
}
```

注意 `acos` 函数对定义域 $[-1, 1]$ 之外的输入会返回 `nan`, 请手动 clip 到 $[-1, 1]$ 的范围内.

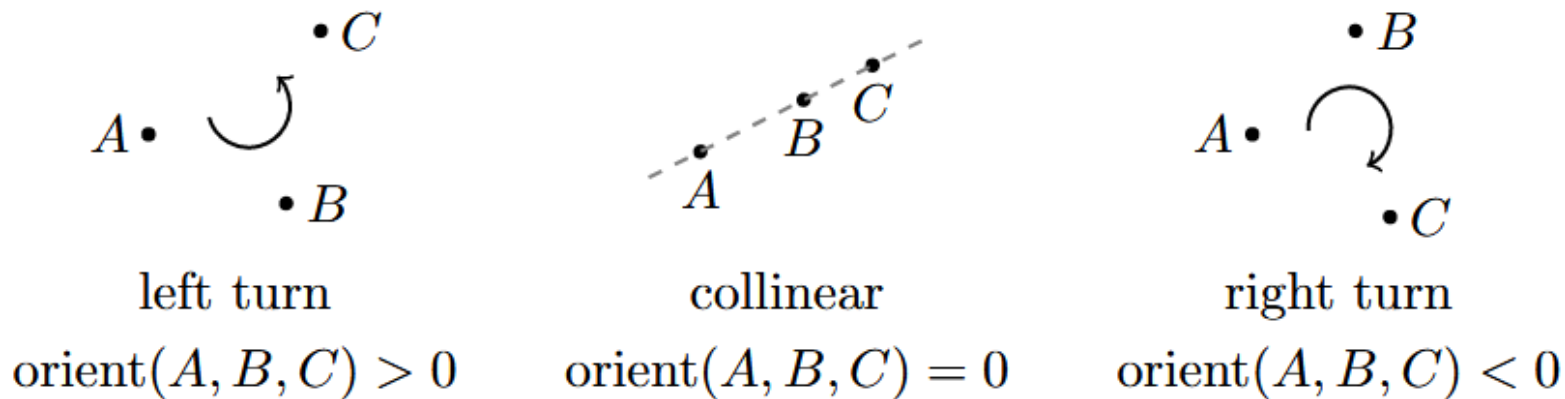
或者使用 `atan2`, 亲测更准一些.

```
T angle(pt v, pt w) {  
    return abs(atan2(cross(v, w), dot(v, w)));  
}
```

2.1 点

叉积的一个重要作用是判断向量之间的相对关系（一个向量在另一个向量的顺时针一侧还是逆时针一侧）。

```
T orient(pt a, pt b, pt c) { return cross(b - a, c - a); }
```

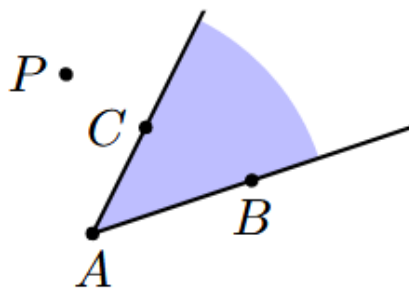


$$\text{orient}(A, B, C) = \text{orient}(B, C, A) = \text{orient}(C, A, B)$$

2.1 点

再利用 $\text{orient}(A, B, C)$ 函数我们可以判断另一个点是否在两个向量所夹的区域内。

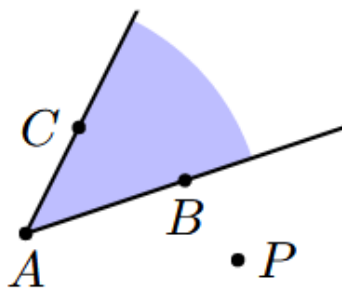
```
bool inAngle(pt a, pt b, pt c, pt p) {  
    assert(orient(a, b, c) != 0);  
    if (orient(a, b, c) < 0) swap(b, c);  
    return orient(a, b, p) >= 0 && orient(a, c, p) <= 0;  
}
```



$$\text{orient}(A, B, P) \geq 0$$

$$\text{orient}(A, C, P) > 0$$

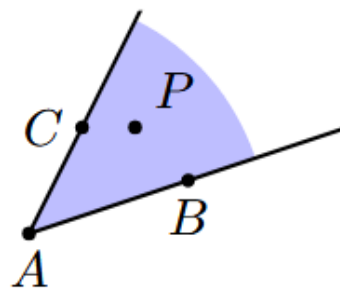
KO



$$\text{orient}(A, B, P) < 0$$

$$\text{orient}(A, C, P) \leq 0$$

KO



$$\text{orient}(A, B, P) \geq 0$$

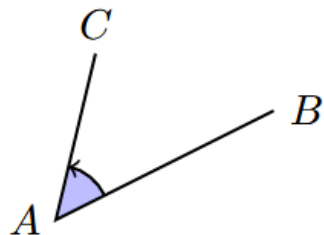
$$\text{orient}(A, C, P) \leq 0$$

OK

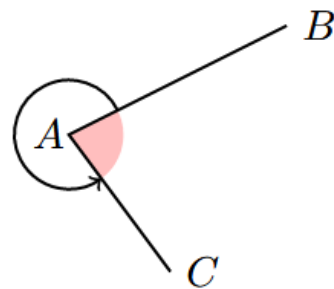
2.1 点

注意我们使用 `angle` 函数求解的角度，可以利用叉积正负性将其升级为带方向的 `orientedAngle` 函数.

```
T orientedAngle(pt a, pt b, pt c) {  
    if (orient(a, b, c) >= 0) { return angle(b - a, c - a); }  
    else { return 2 * M_PI - angle(b - a, c - a); }  
}
```



$\text{orient}(A, B, C) > 0$
 $\text{angle}() = 50^\circ$
 $\text{orientedAngle}() = 50^\circ$

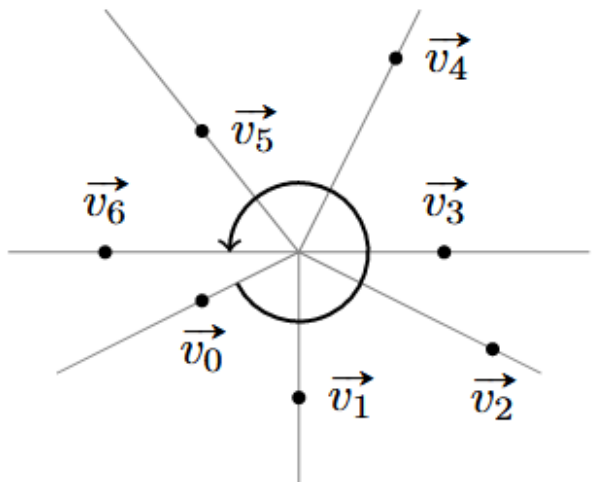


$\text{orient}(A, B, C) < 0$
 $\text{angle}() = 80^\circ$
 $\text{orientedAngle}() = 280^\circ$

2.1 点

2.1.8 极角排序

极角排序是一个经典问题，考察我们是否能以原点为基准，将所有点按照逆时针顺序进行排序。



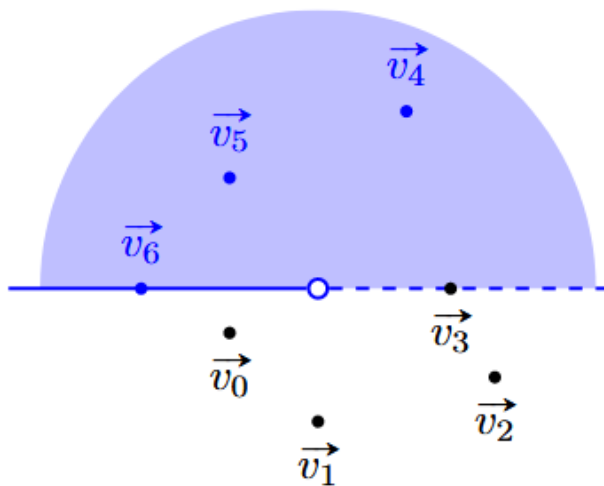
最经典的做法是使用 `atan2` 函数，但其精度可能不够安全……

2.1 点

希望通过叉积将这些向量进行排序.

在 $\vec{v}$ 和 $\vec{w}$ 是相近的向量时 (准确来说, 他们在逆时针顺序上的有向角度 $< 180^\circ$), 则 $\vec{v}$ 在 $\vec{w}$ 前面的充要条件是 $\vec{v} \times \vec{w} > 0$.

如果把所有向量分成到两个半平面上分别做, 则两个半平面内部待排序的角度都不会超过 180° , 就可以正常判断了!



2.1 点

这是一个双关键字排序.

```
bool half(pt p) { // true if in blue half
    assert(p.x != 0 || p.y != 0); // the argument of (0,0) is undefined
    return p.y > 0 || (p.y == 0 && p.x < 0);
}

void polarSort(vector<pt> &v) {
    sort(v.begin(), v.end(), [](pt v, pt w) {
        return make_tuple(half(v), 0, sq(v)) <
               make_tuple(half(w), cross(v,w), sq(w));
    });
}
```

如果两点与原点共线, 则距原点较近的点会被排在前面.

2.2 直线

2.2.1 基本定义

用方向向量 v 和偏移量 c 来表示直线.

```
struct line {  
    pt v; T c;  
    // From direction vector v and offset c  
    line(pt v, T c) : v(v), c(c) {}  
    // From equation ax+by=c  
    line(T a, T b, T c) : v({b, -a}), c(c) {}  
    // From points P and Q  
    line(pt p, pt q) : v(q-p), c(cross(v,p)) {}  
}
```

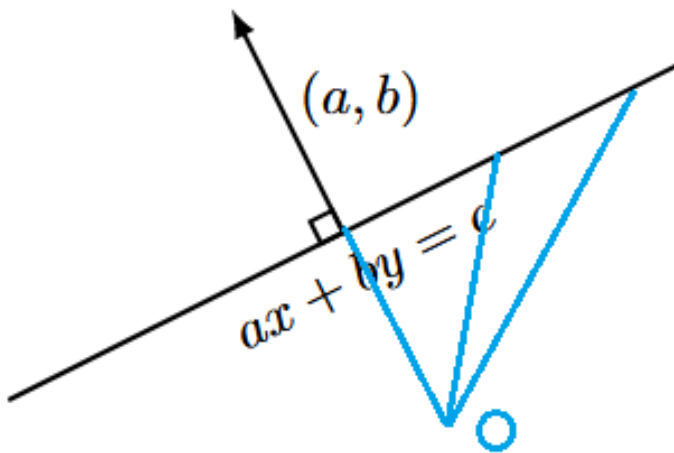
这是怎么来的?

2.2 直线

数学上，我们通常使用 $ax + by = c$ 来表示直线。

取垂直于直线的方向向量 (a, b) ，则直线方程可写成 $(a, b) \cdot (x, y) = c$ 。表示垂直于方向向量 (a, b) 的一系列点。

从原点到这些点的连线到 (a, b) 这一方向向量的投影长度均相同，为 $\frac{c}{\sqrt{a^2 + b^2}}$ 。

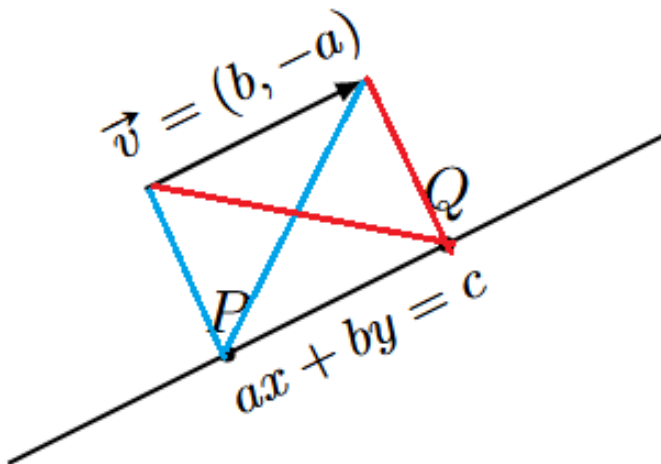


2.2 直线

也可以从叉积的角度理解，取方向向量 $(b, -a)$ ，则直线方程可写成 $(b, -a) \times (x, y) = c$.

其中这个方向向量就是直线的方向向量.

从几何的角度理解，选取 $(0, 0)$ ， $(b, -a)$ 和直线上任一点所构成的三角形面积均相同.



2.2 直线

如果给定直线上两个点 (x_1, y_1) 和 (x_2, y_2) , 想要初始化直线, 则:

$$(b, -a) = (x_2, y_2) - (x_1, y_1)$$

然后因为直线上任一点都满足那个叉积形式的等式:

$$(b, -a) \times (x, y) = (b, -a) \times (x_1, y_1) = c$$

可得:

$$c = (x_2 - x_1, y_2 - y_1) \times (x_1, y_1)$$

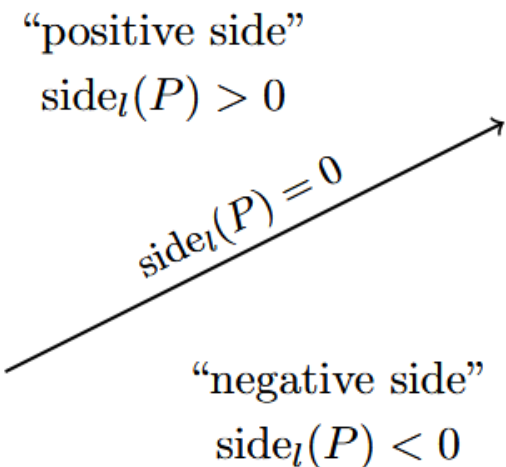
以上, 就是前面给的代码中的三种初始化方式.

2.2 直线

2.2.2 点与直线的位置关系

我们的直线给定了方向向量，据此，我们将直线所划分的两个半平面分别定义为“positive side”和“negative side”。

$$\text{side}_l(P) := ax + by - c = \vec{v} \times P - c$$



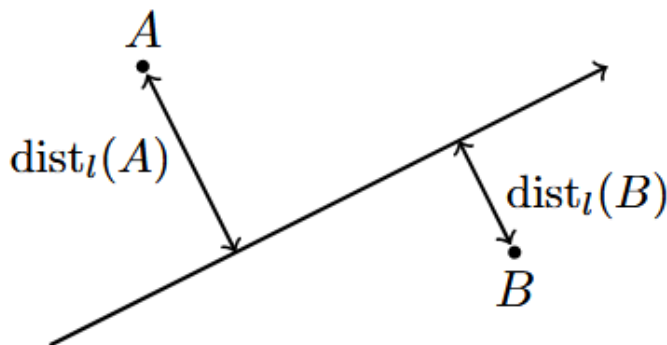
```
T side(line ln, pt p) { return cross(ln.v, p) - ln.c; }
```


2.2 直线

2.2.3 点到直线的距离

如果你熟悉叉积的性质的话，又不难发现：我们可以通过 `side` 函数简单的得到 `dist` 函数。——这就是叉积的性质。

$$\text{dist}_l(P) = \frac{\text{side}_l(P)}{\|\vec{v}\|}$$



```
T dist(line ln, pt p) { return abs(side(ln, p)) / absv(ln.v); }
```

2.2 直线

一个值得注意的细节是，在检查点是否在直线的某个整数距离内时，建议使用 `sqDist` 方法.

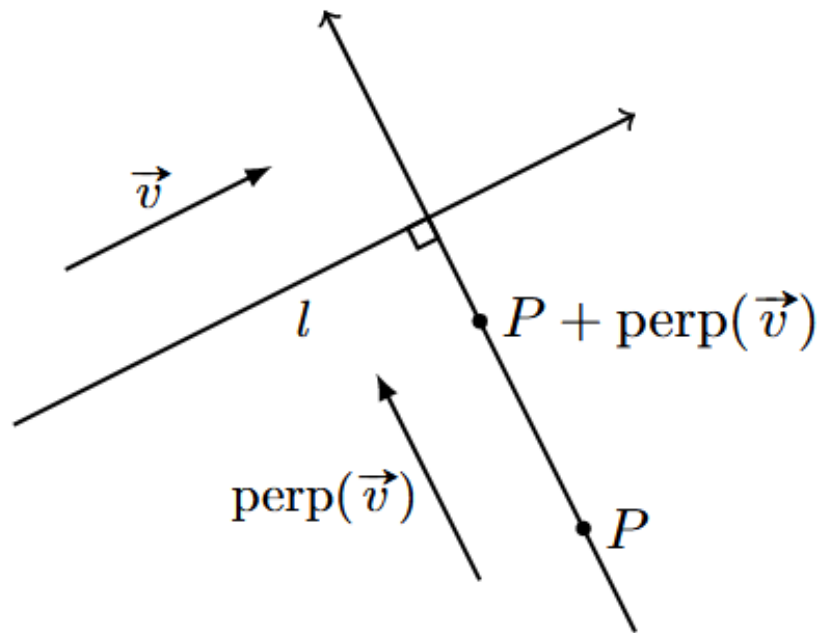
```
T sqDist(line ln, pt p) { return side(ln, p) * side(ln, p) / sq(ln.v); }
```

虽然用到了 `double` ……

回顾我们学到的 IEEE-754 浮点数性质：如果操作数和运算结果都在给定范围内且都是整数，则运算被保证是精确的.

2.2 直线

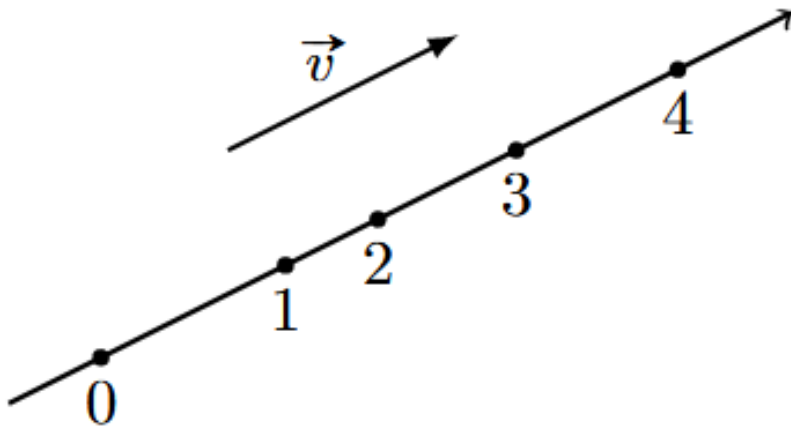
2.2.4 过一点的垂线



```
line perpThrough(line ln, pt p) { return {p, p + perp(ln.v)}; }
```

2.2 直线

2.2.5 沿直线排序

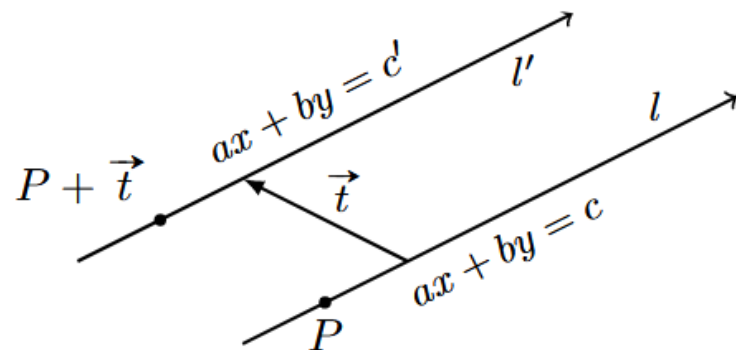


```
bool cmpProj(line ln, pt p, pt q) { return dot(ln.v, p) < dot(ln.v, q); }
```

基于这个比较函数调用 `std::sort` 即可.

2.2 直线

2.2.6 沿指定方向平移



可以用 $\text{side}_l(P)$ 来判断点是否在直线上!

$$\begin{cases} \text{side}_l(P) = \vec{v} \times P - c = 0 \\ \text{side}_{l'}(P + \vec{t}) = \vec{v} \times (P + \vec{t}) - c' = 0 \end{cases} \implies c' = \vec{v} \times (P + \vec{t}) = c + \vec{v} \times \vec{t}$$

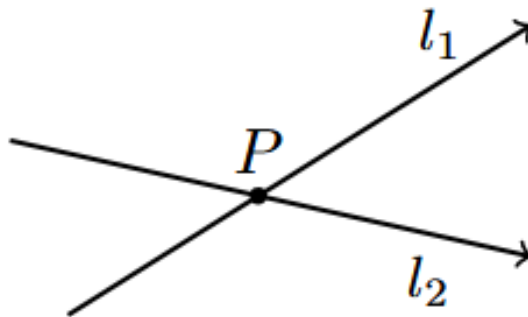
```
line translate(line ln, pt t) { return {ln.v, ln.c + cross(ln.v, t)}; }
```

2.2 直线

2.2.7 直线交点

任意两条不平行 ($\vec{v}_1 \times \vec{v}_2 \neq 0$) 的直线都存在唯一交点:

$$P = \frac{c_1 \vec{v}_2 - c_2 \vec{v}_1}{\vec{v}_1 \times \vec{v}_2}$$



2.2 直线

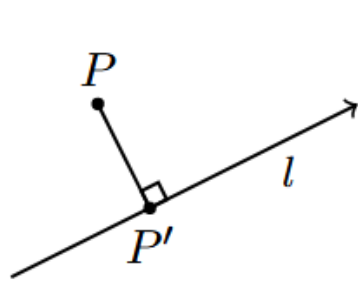
可以用 side 证明:

$$\begin{aligned}\text{side}_{l_1}(P) &= \vec{v}_1 \times \left(\frac{c_1 \vec{v}_2 - c_2 \vec{v}_1}{\vec{v}_1 \times \vec{v}_2} \right) - c_1 \\ &= \frac{c_1(\vec{v}_1 \times \vec{v}_2) - c_2(\vec{v}_1 \times \vec{v}_1) - c_1(\vec{v}_1 \times \vec{v}_2)}{\vec{v}_1 \times \vec{v}_2} \\ &= \frac{-c_2(\vec{v}_1 \times \vec{v}_1)}{\vec{v}_1 \times \vec{v}_2} = 0\end{aligned}$$

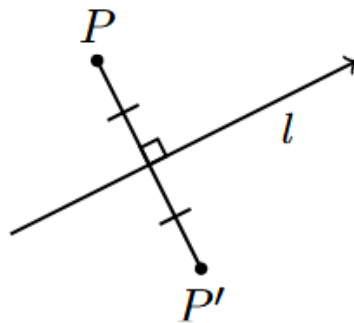
```
bool inter(line l1, line l2, pt &out) {  
    T d = cross(l1.v, l2.v);  
    if (d == 0) return false;  
    out = (l2.v * l1.c - l1.v * l2.c) / d;  
    return true;  
}
```

2.2 直线

2.2.8 正交投影与反射



projection



reflection

相当于求解因子 k , 使得:

$$\text{side}_l(P + k \text{ perp}(\vec{v})) = 0$$

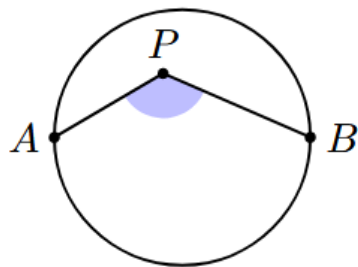
$$\begin{aligned}\text{side}_l(P + k \text{ perp}(\vec{v})) &= \vec{v} \times (P + k \text{ perp}(\vec{v})) - c \\ &= \vec{v} \times P + \vec{v} \times k \text{ perp}(\vec{v}) - c \\ &= (\vec{v} \times P - c) + k(\vec{v} \times \text{perp}(\vec{v})) \\ &= \text{side}_l(P) + k \|v\|^2 \\ \implies k &= -\frac{\text{side}_l(P)}{\|v\|^2}\end{aligned}$$

反射的话就是因子换成 $2k$.

```
pt proj(line ln, pt p) { return p - perp(ln.v) * side(ln, p) / sq(ln.v); }  
pt refl(line ln, pt p) { return p - perp(ln.v) * 2 * side(ln, p) / sq(ln.v); }
```

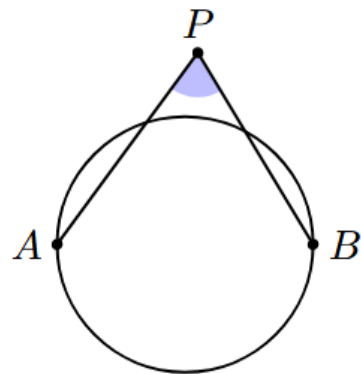
2.3 线段

2.3.1 点在线段为直径的圆上



$$\overrightarrow{PA} \cdot \overrightarrow{PB} \leq 0$$

in disk



$$\overrightarrow{PA} \cdot \overrightarrow{PB} > 0$$

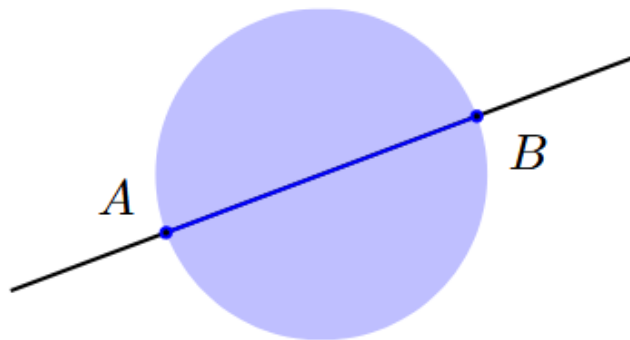
out of disk

```
bool inDisk(pt a, pt b, pt p) { return dot(a - p, b - p) <= 0; }
```

2.3 线段

2.3.2 点在线段上

如果只判断叉积为 0 是不对的，可以用 `inDisk` 函数辅助判断。



intersection of line and disk = segment

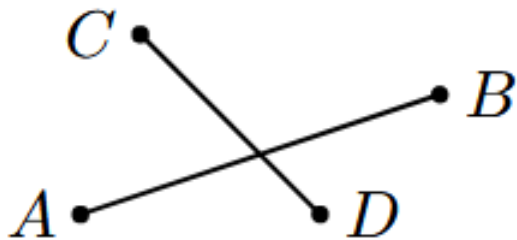
```
bool onSegment(pt a, pt b, pt p) {  
    return orient(a, b, p) == 0 && inDisk(a, b, p);  
}
```

2.3 线段

2.3.3 线段与线段交点

线段与线段之间的交点有两种：

- 一种是正常的：类似于直线相交的情况.
- 另一种是退化的：可能在端点处相交，或者整个线段重合.



proper intersection

2.3 线段

先判断有没有，有的话再通过加权平均（相当于在 A 点的权重为 $1/o_a$ ， B 点的权重为 $1/o_b$ ）得到。

```
bool properInter(pt a, pt b, pt c, pt d, pt &out) {  
    T oa = orient(c, d, a), ob = orient(c, d, b),  
      oc = orient(a, b, c), od = orient(a, b, d);  
    // Proper intersection exists iff opposite signs  
    if (oa * ob < 0 && oc * od < 0) {  
        out = (a * ob - b * oa) / (ob - oa);  
        return true;  
    }  
    return false;  
}
```

2.3 线段

再处理退化的情况.

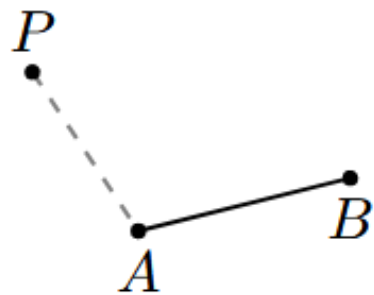
```
set<pt> inters(pt a, pt b, pt c, pt d) {  
    pt out;  
    if (properInter(a, b, c, d, out)) return {out};  
    set<pt> s;  
    if (onSegment(c, d, a)) s.insert(a);  
    if (onSegment(c, d, b)) s.insert(b);  
    if (onSegment(a, b, c)) s.insert(c);  
    if (onSegment(a, b, d)) s.insert(d);  
    return s;  
}
```

如果 $|s| = 1$ 则只有一个交点; 如果 $|s| = 2$ 则说明相交的区域是一条线段, 并且 s 中存的是其两个端点.

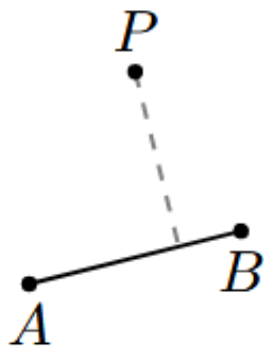
2.3 线段

2.3.4 点到线段的距离

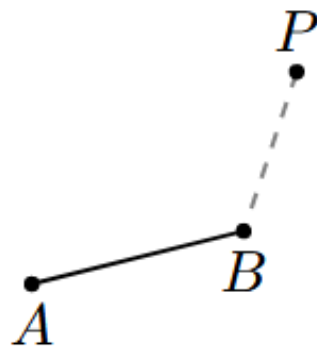
得看垂足有没有落在线段上，否则就是到端点的距离了。



closest to A



closest to projection



closest to B

2.3 线段

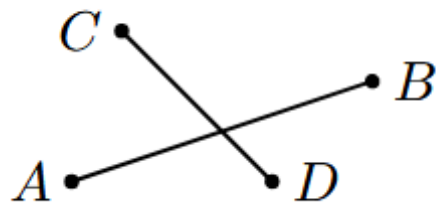
记得处理线段退化成一个点的情况.

```
T segPoint(pt a, pt b, pt p) {  
    if (a != b) {  
        line l(a, b);  
        if (cmpProj(l, a, p) && cmpProj(l, p, b)) // if closest to projection  
            return dist(l, p);                    // output distance to line  
        return min(absv(p - a), absv(p - b));    // otherwise distance to A or B  
    }  
    return absv(p - a);  
}
```

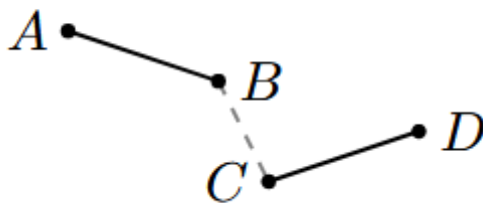

2.3 线段

2.3.5 线段到线段的距离

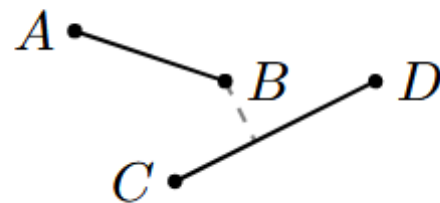
要么直接相交，距离为 0；否则一定可以退化到点到线段距离的 case.



proper intersection



between points

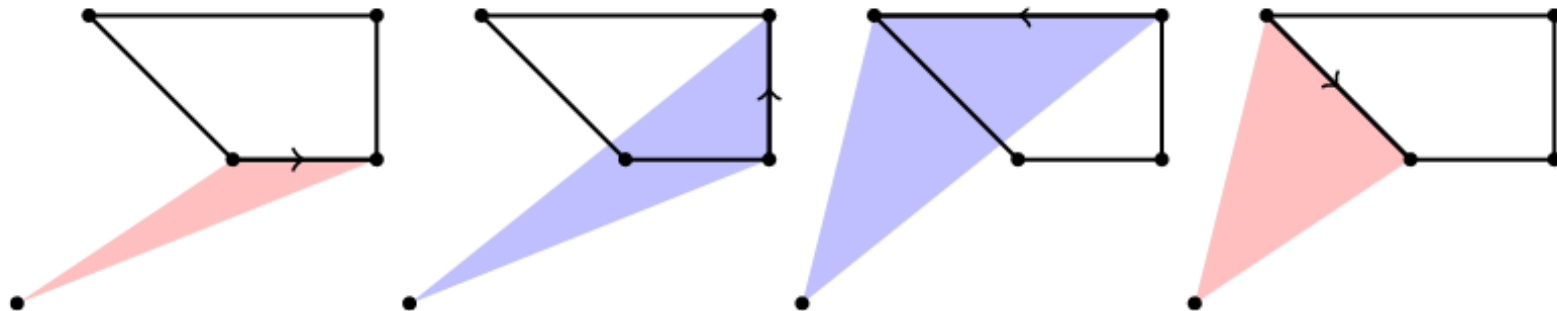


between point and
line

```
T segSeg(pt a, pt b, pt c, pt d) {  
    pt dummy;  
    if (properInter(a, b, c, d, dummy)) return 0;  
    return min({segPoint(a, b, c), segPoint(a, b, d),  
                segPoint(c, d, a), segPoint(c, d, b)});  
}
```

2.4 多边形

2.4.1 多边形面积

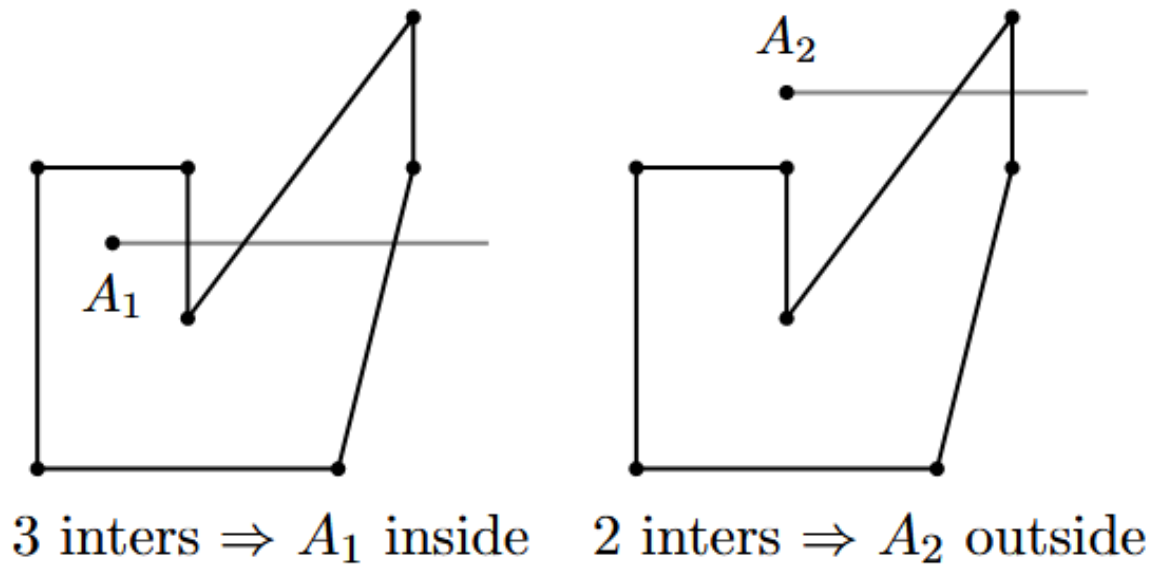


```
T areaPolygon(vector<pt> p) {  
    T area = 0.0;  
    for (int i = 0, n = p.size(); i < n; i++) {  
        area += cross(p[i], p[(i + 1) % n]); // wrap back to 0 if i == n - 1  
    }  
    return abs(area) / 2.0;  
}
```

2.4 多边形

2.4.2 Cutting-ray Test

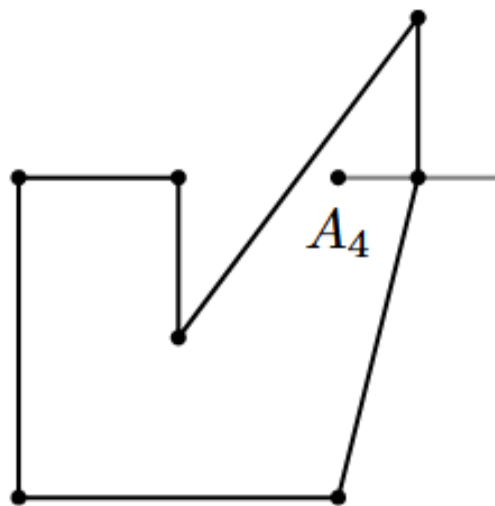
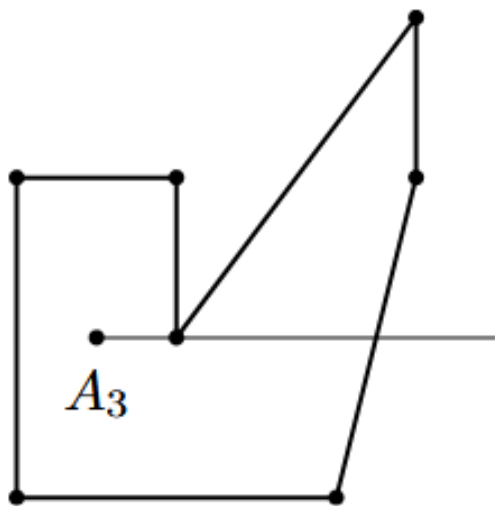
判断点是否在多边形内部的方法：从该点出发引一条直线，数该直线与多边形相交的次数。



2.4 多边形

这个“相交”有很多细节：

端点到底是算还是不算……



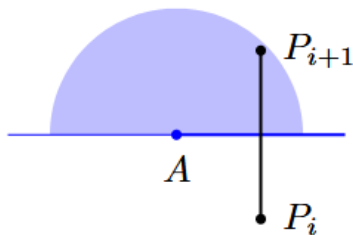
2.4 多边形

1. 线段必须穿过射线.
2. 线段的两个端点必须一个位于半平面内, 一个位于半平面外 (这里的半平面包括 x 轴上的点) .

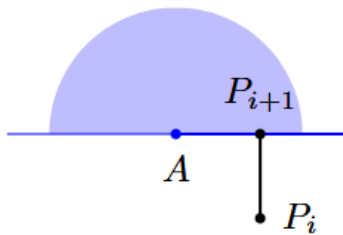
```
// true if P at least as high as A  
(blue part)
```

```
bool above(pt a, pt p) {  
    return p.y >= a.y;  
}
```

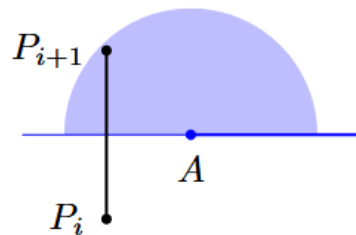
```
// check if [PQ] crosses ray from A  
bool crossesRay(pt a, pt p, pt q) {  
    return (above(a, q) - above(a, p)) *  
           orient(a, p, q) > 0;  
}
```



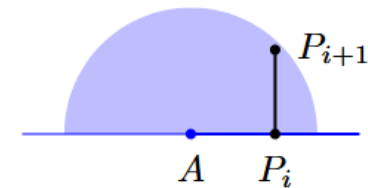
- touches ray: OK
- halves $\neq$: OK
 $\Rightarrow$ crossing



- touches ray: OK
- halves $\neq$: OK
 $\Rightarrow$ crossing



- touches ray: KO
- halves $\neq$: OK
 $\Rightarrow$ no crossing



- touches ray: OK
- halves $\neq$: KO
 $\Rightarrow$ no crossing

2.4 多边形

通过对 `crossesRay` 计数来判断点是否在多边形内. 另外提供了一个额外选线以允许/拒绝点恰好在多边形的边上.

```
// if strict, returns false when A is on the boundary
bool inPolygon(vector<pt> p, pt a, bool strict = true) {
    int numCrossings = 0;
    for (int i = 0, n = p.size(); i < n; i++) {
        if (onSegment(p[i], p[(i + 1) % n], a)) return !strict;
        numCrossings += crossesRay(a, p[i], p[(i + 1) % n]);
    }
    return numCrossings & 1; // inside if odd number of crossings
}
```

另一种方法是使用 Winding Number, 可自行学习.

2.4 多边形

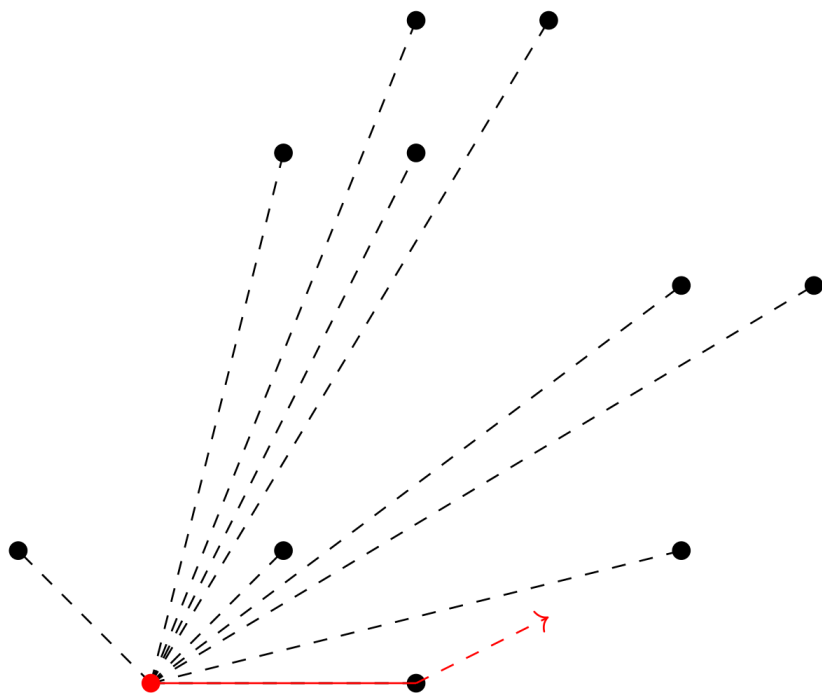
对于已经逆时针排好的凸包的点（CCW），有log级别的判别方式（严格）：

```
bool pointInConvexStrict(const vector<pt>&A, const pt&p) {
    int n = (int)A.size();
    if (n<3) return false;
    if (sgn(orient(A[0], A[1], p)) <= 0) return false;
    if (sgn(orient(A[0], A[n-1], p)) >= 0) return false;
    int l = 1, r = n-1;
    while (r-l > 1) {
        int mid = (l+r) >>1;
        if (sgn(orient(A[0], A[mid], p)) > 0) l = mid;
        else r=mid;
    }
    return sgn(orient(A[l], A[l+1], p)) > 0;}
```

2.4 多边形

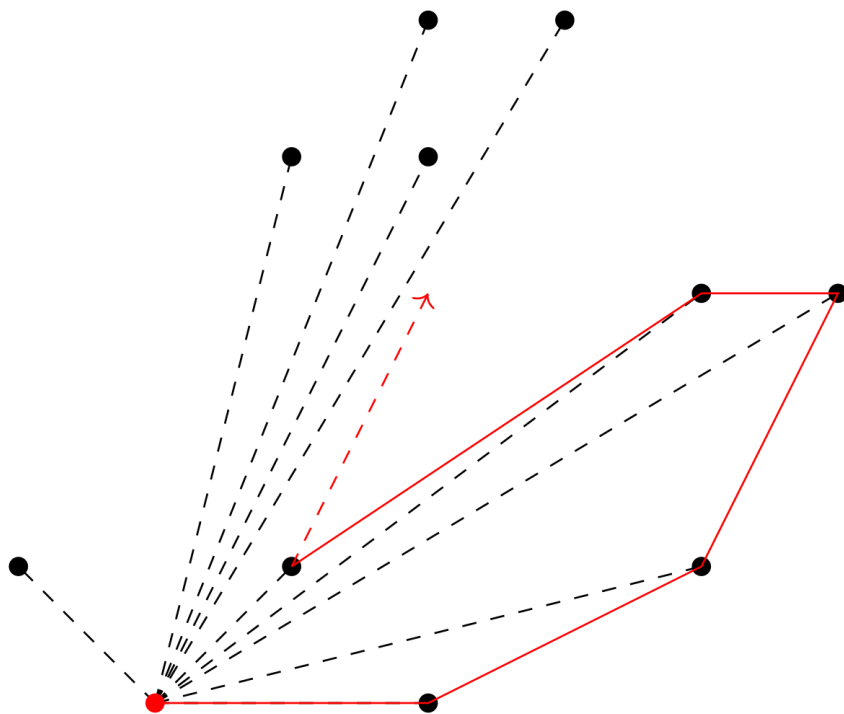
2.4.3 凸包

极角排序后用一个栈维护



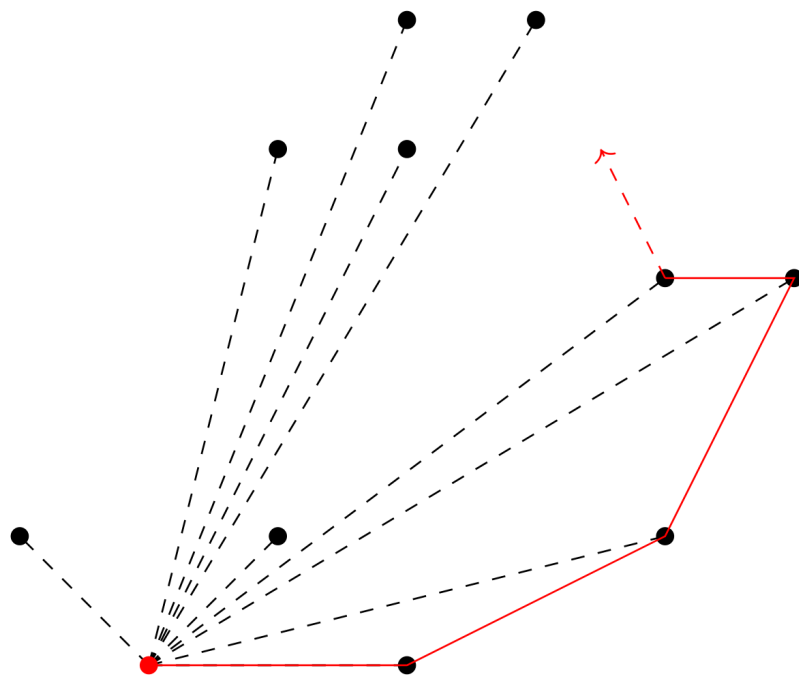
2.4 多边形

按理来说这个凸包应该一直在“turn left”，如果没有的话就从栈里 pop 掉节点.



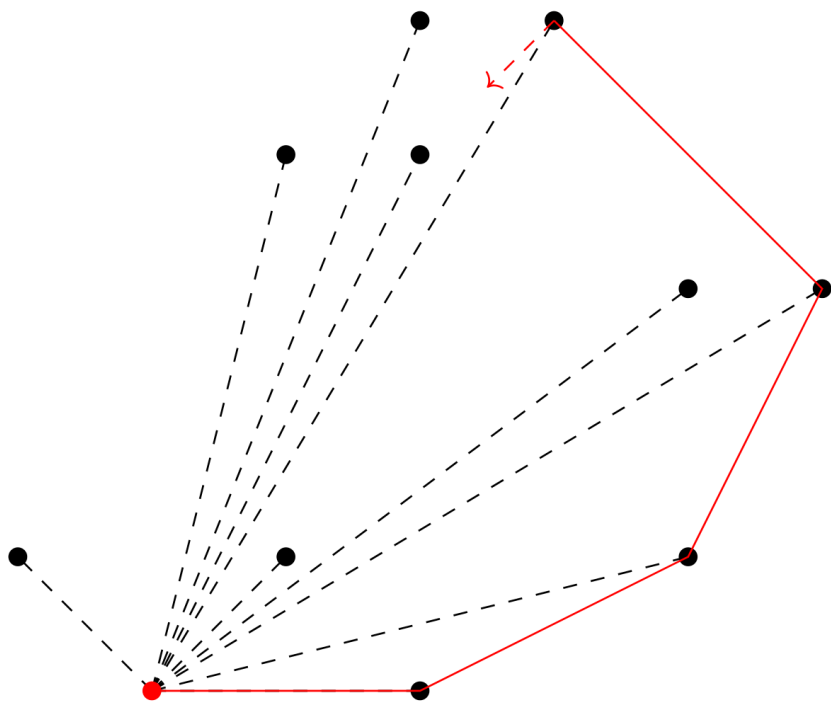
2.4 多边形

可能弹了一个继续不行……



2.4 多边形

这下终于对啦！

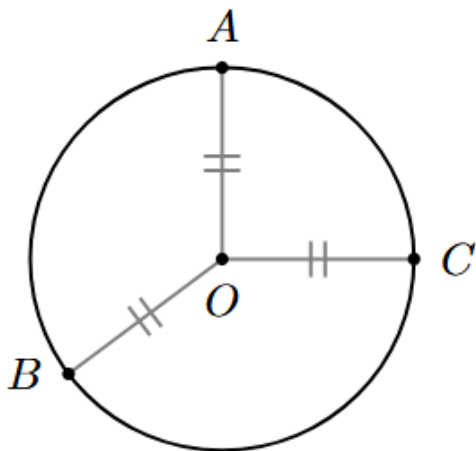


2.4 多边形

```
bool turn_left(pt a, pt b, pt c) { return sgn(cross(b - a, c - a)) > 0; }  
vector<pt> convex_hull(vector<pt> a) {  
    if (a.size() <= 2) return a;  
    pt base = *min_element(a.begin(), a.end());  
    sort(a.begin(), a.end(), [&](auto u, auto v) {  
        return make_tuple(half(u - base), 0) <  
            make_tuple(half(v - base), cross(u - base, v - base));  
    });  
    vector<pt> ret;  
    for (auto i : a) {  
        while (ret.size() > 1 && !turn_left(ret[ret.size()-2], ret[ret.size()-1], i))  
            ret.pop_back();  
        ret.push_back(i);  
    }  
    return ret;  
}
```

2.5 圆

2.5.1 外接圆



外接圆圆心是线段 AB 和 AC 的中垂线的交点.

线段 $|AB|$ 的方向向量是 $\text{perp}(\overrightarrow{AB})$, 中点是 $\frac{1}{2}\overrightarrow{AB}$.

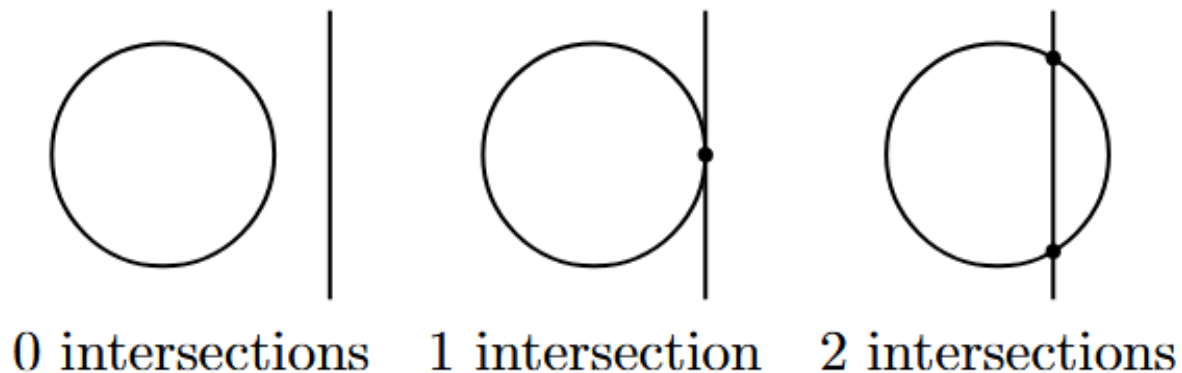
$$\begin{aligned}\overrightarrow{AO} &= \frac{(-\frac{1}{2}|AB|^2) \text{perp}(\overrightarrow{AC}) - (-\frac{1}{2}|AC|^2) \text{perp}(\overrightarrow{AB})}{\text{perp}(\overrightarrow{AB}) \times \text{perp}(\overrightarrow{AC})} \\ &= \frac{\text{perp}(|AC|^2 \overrightarrow{AB} - |AB|^2 \overrightarrow{AC})}{2 \overrightarrow{AB} \times \overrightarrow{AC}}\end{aligned}$$

```
pt circumCenter(pt a, pt b, pt c) {
    b = b - a, c = c - a; // consider coordinates relative to A
    assert(cross(b, c) != 0); // no circumcircle if A,B,C aligned
    return a + perp(b * sq(c) - c * sq(b)) / cross(b, c) / 2;
}
```

2.5 圆

2.5.2 圆与直线的交点

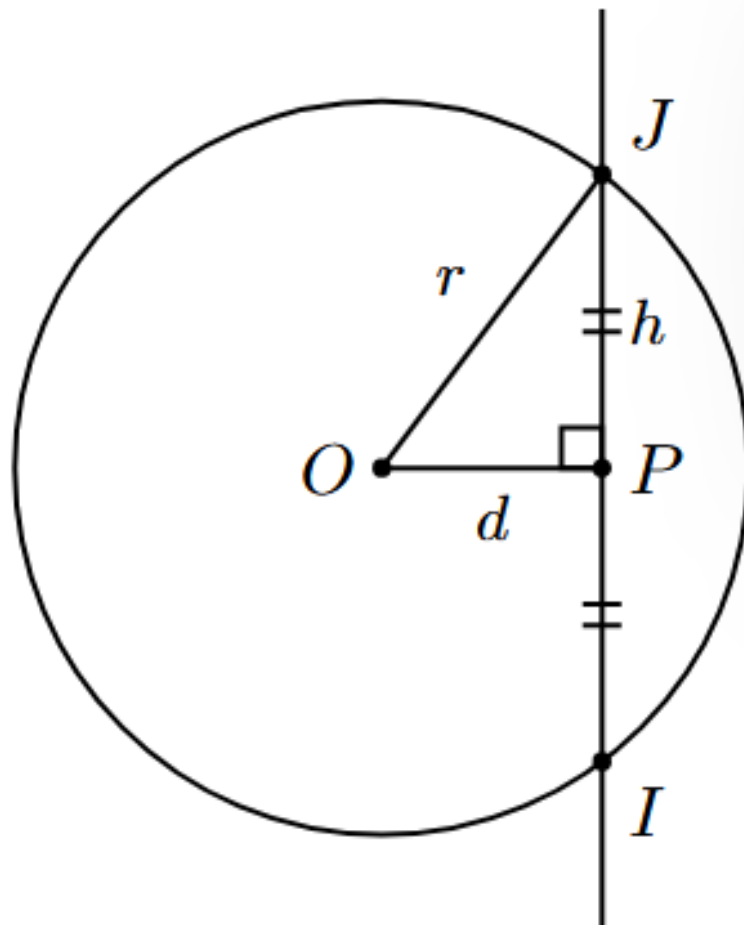
根据圆心到直线的距离就容易判断交点到底有几个.



2.5 圆

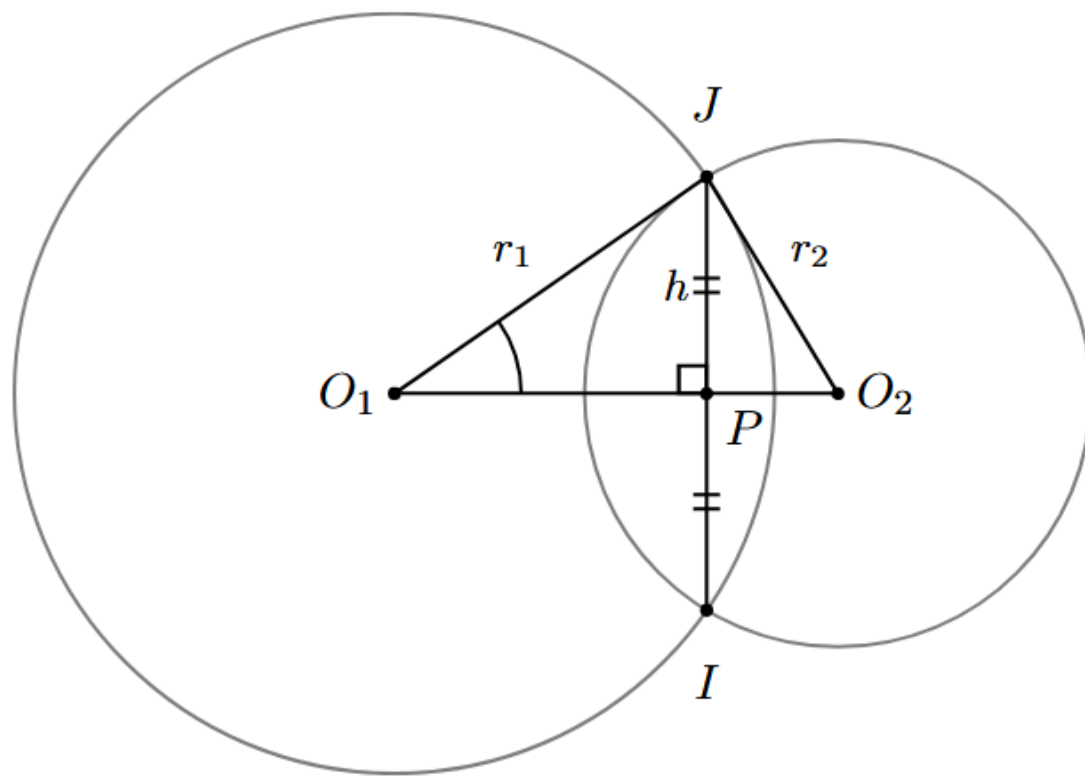
$$h = \sqrt{r^2 - d^2}$$

```
int circleLine(pt o, T r, line ln, pair
<pt, pt> &out) {
    T h2 = r * r - sqDist(ln, o);
    if (h2 >= 0) {
        pt p = proj(ln, o);
        pt h = ln.v * sqrt(h2) / absv(ln.v);
        out = {p - h, p + h};
    }
    return 1 + sgn(h2);
}
```



2.5 圆

2.5.3 圆与圆的交点



Let $d = |O_1O_2|$. We know from the law of cosines on O_1O_2J that

$$\cos(\angle O_2O_1J) = \frac{d^2 + r_1^2 - r_2^2}{2dr_1}$$

and since O_1PJ is a right triangle,

$$|O_1P| = r_1 \cos(\angle O_2O_1J) = \frac{d^2 + r_1^2 - r_2^2}{2d}$$

which allows us to find P .

Now to find $h = |PI| = |PJ|$, we apply the Pythagorean theorem on triangle O_1PJ , which gives $h = \sqrt{r_1^2 - |O_1P|^2}$.

2.5 圆

```
int circleCircle(pt o1, T r1, pt o2, T r2, pair<pt, pt> &out) {
    pt d = o2 - o1;
    T d2 = sq(d);
    if (d2 == 0) {
        assert(r1 != r2);
        return 0; // concentric circles
    }
    T pd = (d2 + r1 * r1 - r2 * r2) / 2; // = |O_1P| * d
    T h2 = r1 * r1 - pd * pd / d2;      // = h^2
    if (h2 >= 0) { pt p = o1 + d * pd / d2, h = perp(d) * sqrt(h2 / d2); out = {p
- h, p + h}; }
    return 1 + sgn(h2);
}
```

- <https://victorlecomte.com/cp-geo.pdf>
- https://fstqwq.pw/Computational_Geometry_2024.pdf